

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



DEVIA DIAN INDRIYANTI

109082500150

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Sorting merupakan proses pengurutan data berdasarkan aturan tertentu, seperti urutan menaik (ascending) maupun menurun (descending). Proses sorting sangat penting dalam pemrograman karena dapat mempermudah pengolahan, pencarian, dan penyajian data agar lebih terstruktur dan efisien.

Pada praktikum ini digunakan metode insertion sort dan selection sort. Insertion sort merupakan metode pengurutan dengan cara menyisipkan elemen ke posisi yang sesuai pada bagian data yang sudah terurut. Proses dilakukan dengan membandingkan elemen saat ini dengan elemen sebelumnya, kemudian memindahkan elemen hingga posisi yang tepat ditemukan.

Selection sort merupakan metode pengurutan dengan cara mencari nilai terkecil atau terbesar pada bagian data yang belum terurut, kemudian menukarkan posisi data tersebut dengan elemen pada indeks tertentu. Proses ini dilakukan secara berulang hingga seluruh data tersusun dengan baik.

Selain algoritma sorting, praktikum ini juga menggunakan konsep array, struct, dan binary search. Array digunakan untuk menyimpan kumpulan data dengan tipe yang sama, struct digunakan untuk mengelompokkan beberapa atribut data dalam satu kesatuan, sedangkan binary search digunakan untuk melakukan pencarian data pada array yang telah terurut secara lebih efisien.

B. Guided

1. Guided 1 – Modul 14

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan bulat
       F.S. array T terurut secara mengecil atau descending dengan
       INSERTION SORT */
}
```

```

var temp, i, j int

i = 1
for i <= n-1 {
    j = i
    temp = T[j]

    for j > 0 && temp > T[j-1] {
        T[j] = T[j-1]
        j = j - 1
    }

    T[j] = temp
    i = i + 1
}

func main() {
    var data arrInt
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSort(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti
\Week 12-Modul 14\Guided\Guided 1\lat1.go"
5
3 7 1 9 2
9 7 3 2 1
PS C:\109082500150_Devia-Dian-Indriyanti>

```

Penjelasan :

Program tersebut menggunakan metode insertion sort untuk mengurutkan data bilangan bulat secara menurun (descending). Proses pengurutan dilakukan dengan membandingkan setiap elemen dengan elemen sebelumnya, kemudian memindahkan posisi elemen sampai berada pada urutan yang sesuai.

Pada program ini, data dimasukkan ke dalam array lalu diproses melalui prosedur insertionSort. Kondisi $temp > T[j-1]$ digunakan agar nilai yang lebih besar berada di depan, sehingga hasil akhir array menjadi terurut dari terbesar ke terkecil.

2. Guided 2 – Modul 14

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrString
    var n int
```

```

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti\Week 12-Modul 14\Guided\Guided 2\lat2.go"
5
jeruk apel mangga pisang kiwi
apel jeruk kiwi mangga pisang
PS C:\109082500150_Devia-Dian-Indriyanti>

```

Penjelasan :

Program tersebut menggunakan metode insertion sort untuk mengurutkan data bertipe string secara ascending atau berdasarkan urutan alfabet. Pengurutan dilakukan dengan membandingkan setiap data dengan data sebelumnya, kemudian memindahkan posisi data sampai berada pada letak yang sesuai.

Pada program ini, data nama atau kata disimpan ke dalam array bertipe string lalu diproses pada prosedur insertionSortString. Kondisi $temp < T[j-1]$ digunakan agar data dengan urutan alfabet lebih kecil ditempatkan di depan, sehingga hasil akhir menjadi terurut dari A ke Z.

3. Guided 3 – Modul 14

```

package main

import "fmt"

func selecetionSortArray(angka *[5]int){
    var idx_min, i, j int

```

```

        for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi
        sortingnya)
            idx_min = i
            for j = i + 1; j < len(angka); j++ { //loop dalam (mencari
            nilai ekstrim)
                if angka[j] <= angka[idx_min] { //sorting terkecil ke
                terbesar
                    idx_min = j
                }
            }
            if idx_min != i {
                angka[i], angka[idx_min] = angka[idx_min], angka[i]
            }
        }
    }

func main(){
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
    selectionSortArray(&arrAngka)
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti
\Week 12-Modul 14\Guided\Guided 3\lat3.go"
Masukkan data angka ke-0 : 3
Masukkan data angka ke-1 : 6
Masukkan data angka ke-2 : 8
Masukkan data angka ke-3 : 9
Masukkan data angka ke-4 : 10

=== SEBELUM DISORTING ===
3 - 6 - 8 - 9 - 10 -
=== SETELAH DISORITNG ===
3 - 6 - 8 - 9 - 10 -
PS C:\109082500150_Devia-Dian-Indriyanti>

```

Penjelasan :

Program tersebut menggunakan metode selection sort untuk mengurutkan data bilangan bulat dari nilai terkecil ke terbesar (ascending). Proses pengurutan dilakukan dengan mencari nilai paling kecil pada bagian array yang belum terurut, kemudian menukarnya dengan posisi data saat ini.

Pada program ini terdapat dua perulangan, yaitu perulangan luar untuk menentukan posisi pengurutan dan perulangan dalam untuk mencari nilai minimum. Jika ditemukan nilai yang lebih kecil, indeks minimum akan diperbarui lalu dilakukan pertukaran data. Setelah seluruh proses selesai, array akan tersusun secara urut dari kecil ke besar.

4. Guided 4 – Modul 14

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK       float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //loop luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam mencari
            nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting terkecil
                ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa
```

```

for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
    fmt.Print("Nama : ")
    fmt.Scan(&structMHS[i].nama)
    fmt.Print("NIM : ")
    fmt.Scan(&structMHS[i].nim)
    fmt.Print("IPK : ")
    fmt.Scan(&structMHS[i].IPK)
}
fmt.Println()

fmt.Println(" === SEBELUM SORITNG === ")
for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM : %s\n", structMHS[i].nim)
    fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
}
fmt.Println("=====")
fmt.Println()

SelectionSortArray(&structMHS)
fmt.Println(" === SETELAH SORITNG === ")
for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM : %s\n", structMHS[i].nim)
    fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
}
fmt.Println("=====")
fmt.Println()
}

```


Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

=== SEBELUM SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : andi
NIM : 63902
IPK : 3.10
--- Data Mahasiswa Ke-1 ---
Nama : budi
NIM : 839374
IPK : 3.50
--- Data Mahasiswa Ke-2 ---
Nama : cimoy
NIM : 83202
IPK : 2.30
--- Data Mahasiswa Ke-3 ---
Nama : dede
NIM : 29101
IPK : 3.40
--- Data Mahasiswa Ke-4 ---
Nama : elon
NIM : 7262910
IPK : 4.00
=====

=== SETELAH SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : cimoy
NIM : 83202
IPK : 2.30
--- Data Mahasiswa Ke-1 ---
Nama : andi
NIM : 63902
IPK : 3.10
--- Data Mahasiswa Ke-2 ---
Nama : dede
NIM : 29101
IPK : 3.40
--- Data Mahasiswa Ke-3 ---
```

Penjelasan :

Program tersebut menggunakan metode selection sort untuk mengurutkan data mahasiswa berdasarkan nilai IPK dari yang terkecil ke terbesar. Data mahasiswa disimpan dalam bentuk struct yang berisi nama, NIM, dan IPK, sehingga proses pengurutan tidak hanya memindahkan satu data, tetapi seluruh informasi mahasiswa sekaligus.

Pada proses sorting, program mencari nilai IPK terkecil pada bagian array yang belum terurut, kemudian menukarkan posisinya dengan data pada indeks saat ini. Setelah proses selesai, data mahasiswa akan tersusun secara ascending berdasarkan IPK. Penggunaan struct pada program ini membuat pengelolaan data menjadi lebih terorganisasi karena setiap atribut mahasiswa disimpan dalam satu kesatuan data.

C. Unguided

1. Soal Unguided 1

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Jawaban :

```
package main

import "fmt"

func main() {
    var arr [100]int
    var n, x int

    for {
        fmt.Scan(&x)

        if x < 0 {
            break
        }

        arr[n] = x
        n++
    }

    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = key
    }

    for i := 0; i < n; i++ {
        fmt.Print(arr[i], " ")
    }
    fmt.Println()
```

```

    if n <= 1 {
        fmt.Println("Data berjarak tidak tetap")
        return
    }

    selisih := arr[1] - arr[0]
    tetap := true

    for i := 2; i < n; i++ {
        if arr[i]-arr[i-1] != selisih {
            tetap = false
            break
        }
    }

    if tetap {
        fmt.Println("Data berjarak", selisih)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti
\Week 12-Modul 14\Unguided\Unguided 1\soal1.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\109082500150_Devia-Dian-Indriyanti>

```

Penjelasan :

Program ini digunakan untuk membaca sekumpulan bilangan bulat non negatif yang diinputkan oleh pengguna. Input akan berhenti ketika pengguna memasukkan bilangan negatif. Setelah itu, data disimpan ke dalam array dan diurutkan menggunakan metode insertion sort secara ascending. Setelah proses pengurutan selesai, program memeriksa apakah selisih antar elemen yang berurutan memiliki jarak yang sama atau tidak. Jika sama, program menampilkan “Data berjarak x”, sedangkan jika berbeda maka menampilkan “Data berjarak tidak tetap”.

2. Soal Unguided 2

- 2) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax ] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Jawaban :

```
package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating    int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)

    for i := 0; i < *n; i++ {
        fmt.Scan(
            &pustaka[i].id,
            &pustaka[i].judul,
            &pustaka[i].penulis,
            &pustaka[i].penerbit,
            &pustaka[i].eksemplar,
            &pustaka[i].tahun,
            &pustaka[i].rating,
        )
    }
}
```

```

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    max := pustaka[0]

    for i := 1; i < n; i++ {
        if pustaka[i].rating > max.rating {
            max = pustaka[i]
        }
    }

    fmt.Println(
        max.judul,
        max.penulis,
        max.penerbit,
        max.tahun,
    )
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    var pass, i int
    var temp Buku

    for pass = 1; pass < n; pass++ {
        temp = pustaka[pass]
        i = pass

        for i > 0 && temp.rating > pustaka[i-1].rating {
            pustaka[i] = pustaka[i-1]
            i--
        }

        pustaka[i] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    batas := 5

    if n < 5 {
        batas = n
    }

    for i := 0; i < batas; i++ {
        fmt.Println(
            pustaka[i].judul,
            pustaka[i].rating,

```

```

    )
}
}

func CariBuku(pustaka DaftarBuku, n, r int) {
    low := 0
    high := n - 1
    found := false

    for low <= high && !found {
        mid := (low + high) / 2

        if pustaka[mid].rating == r {
            found = true

            fmt.Println(
                pustaka[mid].judul,
                pustaka[mid].penulis,
                pustaka[mid].penerbit,
                pustaka[mid].tahun,
                pustaka[mid].eksemplar,
                pustaka[mid].rating,
            )

        } else if r > pustaka[mid].rating {
            high = mid - 1
        } else {
            low = mid + 1
        }
    }

    if !found {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    }
}

func main() {
    var pustaka DaftarBuku
    var n, ratingCari int

    DaftarkanBuku(&pustaka, &n)

    CetakTerfavorit(pustaka, n)

    UrutBuku(&pustaka, n)
}

```

```

    Cetak5Terbaru(pustaka, n)

    fmt.Scan(&ratingCari)

    CariBuku(pustaka, n, ratingCari)
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti
\Week 12-Modul 14\Unguided\Unguided 2\soal2.go"
3
B01 LaskarPelangi Andrea Bentang 10 2005 90
B02 Bumi TereLiye Gramedia 8 2014 95
B03 Dilan Pidi Pastel 7 2014 88
Bumi TereLiye Gramedia 2014
Bumi 95
LaskarPelangi 90
Dilan 88
88
Dilan Pidi Pastel 2014 7 88
PS C:\109082500150_Devia-Dian-Indriyanti>

```

Penjelasan :

Program ini digunakan untuk mengelola data buku pada perpustakaan. Data buku disimpan dalam array bertipe struct Buku. Program dapat menampilkan buku dengan rating tertinggi, mengurutkan buku berdasarkan rating menggunakan insertion sort, menampilkan daftar buku setelah diurutkan, serta mencari buku berdasarkan rating menggunakan binary search.

3. Soal Unguided 3

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort

Jawaban :

```

package main

import "fmt"

```

```
func selectionSort(arr []int) {
    n := len(arr)

    for i := 0; i < n-1; i++ {
        min := i

        for j := i + 1; j < n; j++ {
            if arr[j] < arr[min] {
                min = j
            }
        }

        arr[i], arr[min] = arr[min], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        arr := make([]int, m)

        for j := 0; j < m; j++ {
            fmt.Scan(&arr[j])
        }

        selectionSort(arr)

        for j := 0; j < m; j++ {
            fmt.Print(arr[j])

            if j != m-1 {
                fmt.Print(" ")
            }
        }
        fmt.Println()
    }
}
```


Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti
\Week 12-Modul 14\Unguided\Unguided 3\soal3.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\109082500150_Devia-Dian-Indriyanti>
```

Penjelasan :

Program ini digunakan untuk mengurutkan nomor rumah kerabat Hercules di setiap daerah secara menaik menggunakan algoritma selection sort. Program menerima masukan berupa jumlah daerah dan nomor rumah pada masing-masing daerah, kemudian data diurutkan dari nilai terkecil ke terbesar. Algoritma selection sort bekerja dengan mencari nilai terkecil pada array, lalu menukarnya ke posisi depan secara berulang hingga seluruh data terurut.

4. Soal Unguided 4

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Jawaban :

```
package main

import "fmt"

func main() {
    var daerah int
    fmt.Scan(&daerah)

    for i := 0; i < daerah; i++ {

        var n int
        fmt.Scan(&n)
```

```

data := make([]int, n)

for j := 0; j < n; j++ {
    fmt.Scan(&data[j])
}

var ganjil []int
var genap []int

for j := 0; j < n; j++ {
    if data[j]%2 == 1 {
        ganjil = append(ganjil, data[j])
    } else {
        genap = append(genap, data[j])
    }
}

for j := 0; j < len(ganjil)-1; j++ {
    for k := j + 1; k < len(ganjil); k++ {
        if ganjil[j] > ganjil[k] {
            ganjil[j], ganjil[k] = ganjil[k], ganjil[j]
        }
    }
}

for j := 0; j < len(genap)-1; j++ {
    for k := j + 1; k < len(genap); k++ {
        if genap[j] < genap[k] {
            genap[j], genap[k] = genap[k], genap[j]
        }
    }
}

for j := 0; j < len(ganjil); j++ {
    fmt.Print(ganjil[j], " ")
}
fmt.Println()

for j := 0; j < len(genap); j++ {
    fmt.Print(genap[j], " ")
}
fmt.Println()
}

```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti\Week 12-Modul 14\Unguided\Unguided 4\lat4.go"
3
5 2 1 7 9 13
1 7 9 13
2
6 189 15 27 39 75 133
15 27 39 75 133 189

3 4 9 1
1 9
4
PS C:\109082500150_Devia-Dian-Indriyanti> |
```

Penjelasan :

Program ini digunakan untuk memisahkan data bilangan ganjil dan genap. Bilangan ganjil diurutkan dari kecil ke besar, sedangkan bilangan genap diurutkan dari besar ke kecil menggunakan proses sorting sederhana. Setelah diurutkan, hasil data ganjil dan genap ditampilkan secara terpisah.

D. Kesimpulan

Berdasarkan praktikum modul 14 tentang sorting, dapat disimpulkan bahwa algoritma sorting digunakan untuk mengurutkan data agar lebih terstruktur dan mudah diproses. Metode insertion sort bekerja dengan menyisipkan data pada posisi yang sesuai, sedangkan selection sort bekerja dengan mencari nilai minimum atau maksimum kemudian menukarkan posisinya secara bertahap hingga data terurut.

Pada praktikum ini, proses sorting diterapkan pada berbagai jenis data seperti bilangan bulat, string, dan struct mahasiswa. Selain itu, digunakan juga metode binary search untuk mencari data pada array yang telah terurut. Dengan praktikum ini, pemahaman mengenai penggunaan algoritma sorting, array, struct, dan proses pencarian data menjadi lebih baik serta dapat diterapkan dalam penyelesaian berbagai permasalahan pemrograman.